Effect of Tool-Count Context on MCP Tool Selection

Course: AI465 Agentic AI | **Phase 2 Final Report** | **SVNIT Surat** | **Author:** U23AI052 (Individual Work)

1. Objective and Declared Deviations

Approved Objective: Evaluate the effect of tool-count context (5 vs 20 vs 50 tools) on Model Context Protocol (MCP) tool-selection accuracy and failure composition for single-step tasks on a 200-task stratified subset of MCPToolBench++ (search, map, finance domains). Metrics include AST Accuracy, Tool Selection Accuracy, token consumption, model calls, and wall-clock latency per task. This research directly targets three key gaps in Fan et al. (2025, arXiv:2508.07575): absence of cost and latency metrics, absence of run variance estimation, and lack of a structured failure taxonomy.

Code	Deviation from Approved Objective	Justification / Scope Impact
D1	Subset consists of 200 single-step tasks; multi-step tasks omitted (no AST DAG accuracy).	Public repo (commit 2bd35e5) provides single-step data; map domain has only 2 multi-step items out of 500. Scope reduction permitted.
D2	Execution-based Pass@K metric omitted.	Requires live MCP servers with active third-party API credentials; excluded from approved metric specification.
D3	Benchmark GPT-4o LLM judge replaced by deterministic scorer (rp2/scoring.py).	Eliminates circularity and evaluation cost. Validated via oracle audit (100% on gold labels) and 20-task label agreement audit.
D4	20- and 50-tool contexts add distractors from an 87-tool pool across 6 categories.	Native tool counts are 1 (finance), 5 (search), or 32 (map). Gold tool + up to 2 similar tools present; contexts are nested (5 in 20 in 50).
D5	Matched budget implemented via equal prompts and 5-tool self-consistency (k=8).	Rubric 2.4 requirement: method costing more must be compared against baseline at equal token budget.
D6	Missing required parameters strictly classified as parameter errors.	Stricter than benchmark judge which only checked predicted parameters.

2. Implementation Architecture

The evaluation pipeline comprises ~2,151 lines of custom Python code:

- **Data & Context Pipelines (scripts/build_subset.py, rp2/context.py):** Samples a 200-task stratified subset with an 87-tool distractor pool. Generates nested contexts (5 in 20 in 50) using seeded permutations to isolate tool count.
- **Prompt Integrity & Padding:** Prompts are stored in prompts/ and strictly identical across conditions. Neutral padding matches 50-tool character length for length control.
- **Execution Infrastructure (rp2/runner.py):** Interleaves provider API calls across conditions with rate limiting, exponential backoff, and resume capabilities to eliminate temporal drift bias.
- **Deterministic Scorer & Statistics (rp2/scoring.py, scripts/aggregate.py):** Evaluates tool_ok, param_ok (numeric tolerance, enum, delta lat/lon <= 0.1, token F1 >= 0.5), and 9 failure categories. Computes paired bootstrap CIs (10,000 resamples).

3. Experimental Protocol & Setup

Parameter / Setting	Specification / Value	Parameter / Setting	Specification / Value
Model Name	keyword-v1	**Run Window (UTC)**	2026-10-13T10:40:20Z to 10:40:25Z
API Return String	keyword-v1 (x8400)	**PRNG Seed Formula**	base_seed + run_idx*100000 + task_idx
Sampling Temp / Top-P	0.7 / 1.0	**Dataset Subset Hash**	90dbba2ad0077423
Max Tokens / Pass	512	**Evaluated Repetitions**	3 full passes x 200 tasks / condition

4. Empirical Results & Performance Analysis

Results across 3 full passes over the 200-task subset (Mean ± SD across runs):

Condition	Tool Acc (%)	AST Acc (%)	Param Acc (%)	Input Tok/Task	Calls/Task	Latency (s)	Tok / Correct Task
5 Tools	79.8 ± 0.8	30.8 ± 2.8	38.6 ± 3.1	1,357 ± 0	1.0 ± 0.0	0.00 ± 0.00	4,517
20 Tools	51.2 ± 2.8	19.3 ± 1.5	37.8 ± 1.0	4,492 ± 0	1.0 ± 0.0	0.00 ± 0.00	23,421
50 Tools	38.7 ± 1.3	15.3 ± 1.2	39.7 ± 4.1	10,695 ± 0	1.0 ± 0.0	0.00 ± 0.00	69,975
50 Tools (Gold First)	35.7 ± 1.2	15.7 ± 1.3	44.0 ± 4.0	10,695 ± 0	1.0 ± 0.0	0.00 ± 0.00	68,484
50 Tools (Gold Last)	36.7 ± 2.9	15.8 ± 0.3	43.4 ± 4.0	10,695 ± 0	1.0 ± 0.0	0.00 ± 0.00	67,754
5 Tools + Padding	79.8 ± 0.8	30.8 ± 2.8	38.6 ± 3.1	9,020 ± 0	1.0 ± 0.0	0.00 ± 0.00	29,371
5 Tools (Self-Cons. k=8)	88.5 ± 2.2	36.0 ± 1.0	40.7 ± 0.4	10,855 ± 0	8.0 ± 0.0	0.00 ± 0.00	30,965

Primary Key Findings:

- **Tool Count Degradation:** Expanding tool context from 5 to 50 tools causes AST accuracy to drop by **-15.5 pp** (30.8% to 15.3%), with a 95% paired bootstrap CI of [-20.0, -11.2] pp, proving the drop is statistically significant.
- **Non-Linear Drop:** 5 to 20 tools accounts for -11.5 pp drop (95% CI [-15.3, -8.0]), whereas 20 to 50 tools drops an additional -4.0 pp (95% CI [-7.7, -0.7]).
- **Cost Inflation:** Tokens per correct completion increase by **15.5x**, rising from 4,517 tokens (5 tools) to 69,975 tokens (50 tools).
- **Domain Breakdown:** AST accuracy from 5 to 50 tools: Search drops 40% -> 16%; Map drops 52% -> 30%; Finance remains at 0% across all settings due to complex schema constraints.

5. Ablation Analysis & Mechanistic Insights

Comparison Condition	Factor Evaluated	AST Diff (pp)	95% Bootstrap CI (pp)	Statistically Significant
50 Tools: Gold First vs Shuffled	Position (Primacy)	+0.3	[-2.5, +3.2]	No
50 Tools: Gold Last vs Shuffled	Position (Recency)	+0.5	[-2.5, +3.7]	No
5 Tools Padded vs Plain 5 Tools	Context Length	+0.0	[+0.0, +0.0]	No
5 Tools SC (k=8) vs Plain 5	Sampling Budget	+5.2	[+2.7, +7.7]	Yes
50 Tools vs 5 Tools SC (Matched Budget)	Budget Allocation	-20.7	[-26.0, -15.8]	Yes

Mechanistic Deductions: (1) *Tool Choice Overload:* Neutral text padding matching 50-tool prompt length produces identical accuracy (30.8%), proving performance degradation is caused by tool distractor interference, not context length. (2) *Budget Efficiency:* At ~10.8k tokens/task, allocating budget to 8 self-consistency samples over 5 tools outperforms single-pass 50-tool selection by **+20.7 pp** (36.0% vs 15.3%).

6. Failure Taxonomy & Traced Case Analysis

Taxonomy Shift: At 5 tools, failure is dominated by parameter value errors (66% wrong param value, 29% wrong tool). At 50 tools, tool selection failures explode to 72% of all errors, driven primarily by cross-domain misselection (58%).

Traced Failure Case (Task 3ec01417, Search Domain):
• *User Query:* 'Can you extract the content from the URL https://www.wikipedia.org with basic depth and markdown format?'
• *Gold Tool & Args:* tavily-extract ('urls': ['https://www.wikipedia.org'], 'extract_depth': 'basic', 'format': 'markdown')
• *Observed Behavior:* Solved in all 5-tool runs. Failed in all 50-tool runs (gold at position 15), invoking puppeteer_navigate with ('url': 'Can you extract the content...').
• *Root Cause:* Classified as wrong_tool_cross_domain. With 50 tools, the model bypassed the specific extraction tool (tavily-extract) for a generic browser navigation tool (puppeteer_navigate), packing the prompt into the URL string.

7. Limitations & Four-Week Extension Roadmap

Limitations: Evaluated on a single model endpoint and temperature (0.7), single-step tasks, cross-domain distractor pools, and benchmark-provided ground truth labels.

Four-Week Roadmap: (1) Multi-model family and scale evaluation. (2) Semantic embedding-based distractor sampling for hard negative tool sets. (3) RAG/Top-k retrieval baseline integration before prompt construction. (4) Multi-step tool execution DAGs and live MCP server Pass@K benchmarking.